Dados Relacionais com Polars

Isabela Diniz Cia

Objetivo

Dados relacionais são uma constante no exercício da profissão do estatístico. Esta estratégia permite uma representação mais efetiva de dados estruturados, oferecendo a possibilidade de análises computacionalmente mais eficientes.

Os Dados

O banco de dados para esta atividade é o "IMDb Movie Data", que possui informações sobre filmes, atores, diretores, gêneros e outros. Para os arquivos indicados abaixo, considere que o símbolo `\N` representa valores faltantes. Observe com cuidado a extensão dos arquivos para uma indicação do formato do mesmo.

```
library(reticulate)
py_require("polars")
```

```
import polars as pl
```

Tabela basics

Informações básicas dos títulos

Uma linha por título (filme, série, curta etc.).

- `tconst`: código único do título (ex.: `tt0111161`). É a chave que liga as tabelas.
- `titleType`: tipo do título (`movie`, `tvSeries`, `short`…).
- `primaryTitle` / `originalTitle`: título principal e título original.
- `isAdult`: indica se é conteúdo adulto (0 ou 1).
- `startYear` / `endYear`: ano de lançamento e, para séries, ano de término.
- `runtimeMinutes`: duração em minutos.
- `genres`: até 3 gêneros em uma única string separada por vírgula (ex.: `Drama`, `Romance`).

```
basics = pl.scan_csv(
    "title.basics0.tsv.gz",
    separator="\t",
    null_values="\\N",
    quote_char=None,
    infer_schema_length=10000
    )
```

Tabela ratings

Avaliações

Uma linha por título que tem nota.

- `tconst`: código do título (liga com `basics` e `principals`).
- `averageRating`: nota média dos usuários.
- `numVotes`: quantidade de votos que geraram essa nota.

```
ratings = pl.scan_csv(
    "title.ratings.tsv.gz",
    separator="\t",
    null_values="\\N",
    quote_char=None
    ).with_columns(
        pl.col("averageRating").cast(pl.Float64),
        pl.col("numVotes").cast(pl.Int64)
    )
```

Tabela principals

Elenco e equipe principal

Uma linha por pessoa em cada título, ou seja, cada título aparece várias vezes.

- `tconst`: código do título.
- `ordering`: ordem em que a pessoa aparece na lista do título.
- `nconst`: código único da pessoa (ator, diretor etc.).
- `category`: função da pessoa (`actor`, `actress`, `director`, `writer`…).
- `job`: detalhe do cargo, quando existe.
- `characters`: nome do personagem, quando é ator/atriz.

```
principals = pl.scan_csv(
    "title.principals0.tsv.gz",
    separator="\t",
    null_values="\\N",
    quote_char=None
    )
```

Atividade

Utilizando apenas operações por meio da biblioteca polars, responda:

1. Quais são os 5 filmes com as maiores notas (averageRating)? Apresente uma solução capaz de desempatar os filmes baseando-se no número de votos recebidos.

```
filmes_aval = basics.join(ratings, on="tconst", how="inner")
top5 = (
  filmes_aval
  .filter(pl.col("titleType") == "movie")
  .sort(
    ["averageRating", "numVotes"],
    descending=[True, True]
    )
  .select(
    "primaryTitle",
    "startYear",
    "averageRating",
    "numVotes"
    )
  .head(5)
)

print(top5.collect(engine="streaming"))
```

```
shape: (5, 4)

 primaryTitle              startYear   averageRating   numVotes
 ---                       ---         ---             ---
 str                       i64         f64             i64
```

```
Kaveri                      2024        10.0            1023
Kurukku                     2024        10.0            451
Jedal Dar Omghe 30 Metri    2009        10.0            142
Sargashte                   2016        10.0            134
Gorgeous Rascal             2024        10.0            115
```

2. Qual é o gênero mais frequente entre os filmes com nota maior que 8?

```
top_generos = (
  basics
  .join(ratings, on="tconst", how="inner")
  .filter(
      (pl.col("titleType") == "movie") &
      (pl.col("averageRating") > 8)
      )
  .select("genres")
  .filter(pl.col("genres").is_not_null())
  .with_columns(
      pl.col("genres").str.split(",")
      )
  .explode("genres")
  .filter(pl.col("genres") != "")
  .group_by("genres")
  .agg(
      pl.len().alias("frequencia")
      )
  .sort(
      ["frequencia", "genres"],
      descending=[True, False]
      )
  .head(1)
)

print(top_generos.collect(engine="streaming"))
```

```
shape: (1, 2)

 genres       frequencia
 ---          ---
```

```
  str           u32

  Documentary   10564
```

3. Quais são os 3 atores/atrizes que mais participaram de filmes com nota maior que 7.5?

```
top3 = (
  principals
  .filter(
    pl.col("category").is_in(["actor", "actress"])
    )
  .join(
    basics.filter(pl.col("titleType") == "movie"),
    on="tconst",
    how="inner"
    )
  .join(
    ratings,
    on="tconst",
    how="inner"
    )
  .filter(pl.col("averageRating") > 7.5)
  .select("tconst", "nconst")
  .unique(subset=["tconst", "nconst"])
  .group_by("nconst")
  .agg(pl.len().alias("nfilmes"))
  .sort(
    ["nfilmes", "nconst"],
    descending=[True, False]
    )
  .head(3)
)
print(top3.collect(engine="streaming"))
```

```
shape: (3, 2)

  nconst      nfilmes
  ---         ---
  str         u32
```

```
  nm0004660   199
  nm0595934   155
  nm3183374   124
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-10-14 21:45:01 -03"
```